“‘

École Nationale des Sciences Appliquées

Filière : IA & Cybersécurité

Compte Rendu de TP

Architecture SOA Spring Boot

Réalisé par : KAYA Hind
Encadré par : Pr. EL BAGHZAOUI

Année universitaire : 2025 - 2026 “‘

Table des matières

1 Introduction **2**

2 Mise en place du serveur Eureka **2**

3 Implémentation du Order Service **3**

4 Conclusion **6**

1 Introduction

Ce TP a pour objectif de mettre en place une architecture microservices en utilisant Spring Boot. Nous avons utilisé Eureka pour la découverte des services et OpenFeign pour la communication entre les microservices.

2 Mise en place du serveur Eureka

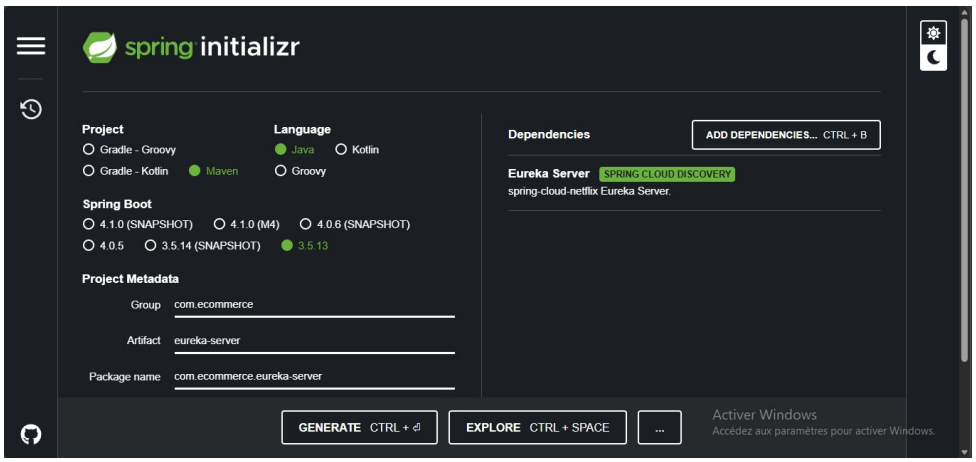


FIGURE 1 – Création du projet Eureka Server via Spring Initializr. Cette étape permet de générer la structure de base du projet avec les dépendances nécessaires.

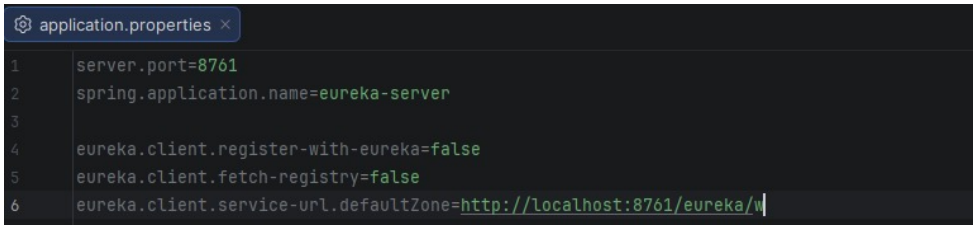
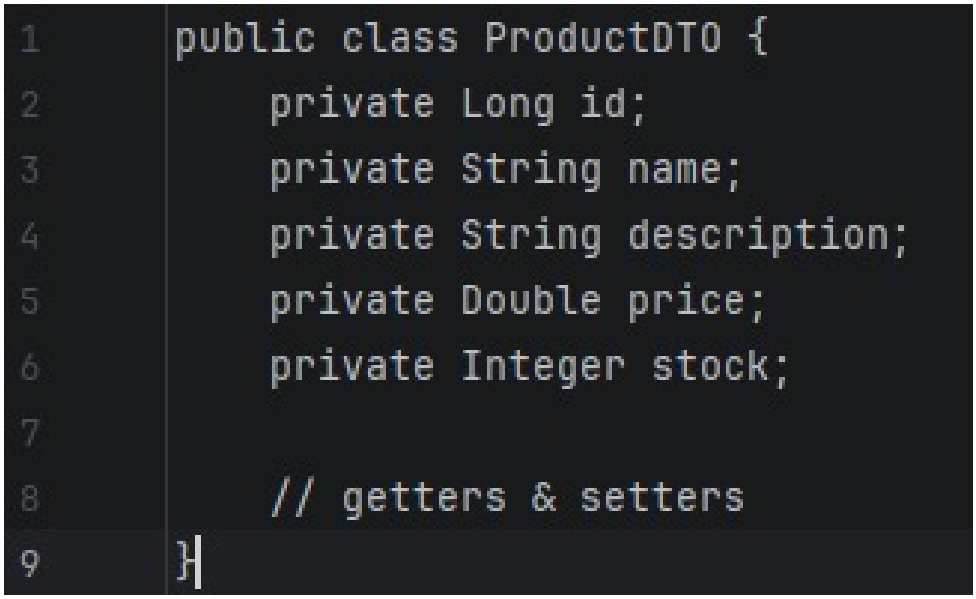


FIGURE 2 – Configuration du fichier application.properties pour Eureka. On définit le port et les paramètres nécessaires pour le fonctionnement du serveur.

FIGURE 3 – Interface Eureka après lancement. Cette page montre que le serveur fonctionne correctement et qu'il est prêt à enregistrer les services.

3 Implémentation du Order Service



```
public class ProductDTO {
    private Long id;
    private String name;
    private String description;
    private Double price;
    private Integer stock;

    // getters & setters
}
```

FIGURE 4 – Classe ProductDTO utilisée pour représenter les données du produit reçues depuis Product Service.

```java
@FeignClient(name = "product-service")
public interface ProductClient {

    @GetMapping("/api/products/{id}")
    ProductDTO getProduct(@PathVariable Long id);
}
```

FIGURE 5 – Feign Client permettant à Order Service de communiquer avec Product Service en appelant ses APIs.

```java
@Entity
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue
    private Long id;

    private Long productId;
    private String productName;
    private Integer quantity;
    private Double totalPrice;
}
```

FIGURE 6 – Classe Order (modèle) représentant une commande avec les informations du produit et le prix total.

```java
orderRepository ×
public interface OrderRepository extends JpaRepository<Order, Long> {}
```

FIGURE 7 – OrderRepository utilisé pour accéder à la base de données et gérer les opérations CRUD sur les commandes.

```java
orderservice
@Service
public class OrderService {

    @Autowired
    private OrderRepository repo;

    @Autowired
    private ProductClient productClient;

    public Order createOrder(Long productId, int quantity) {

        ProductDTO product = productClient.getProduct(productId);

        Order o = new Order();
        o.setProductId(productId);
        o.setProductName(product.getName());
        o.setQuantity(quantity);
        o.setTotalPrice(product.getPrice() * quantity);

        return repo.save(o);
    }
}
```

FIGURE 8 – Service OrderService contenant la logique métier : appel au Product Service, calcul du prix total et sauvegarde de la commande.

```java
orderController
@RestController
@RequestMapping("/api/orders")
public class OrderController {

    @Autowired
    private OrderService service;

    @PostMapping
    public Order create(@RequestParam Long productId,
                        @RequestParam int quantity) {
        return service.createOrder(productId, quantity);
    }
}
```

FIGURE 9 – OrderController exposant les APIs REST pour créer une commande en utilisant les paramètres productId et quantity.

4 Conclusion

Dans ce TP, nous avons mis en place une architecture microservices basée sur Spring Boot. Nous avons utilisé Eureka pour la découverte des services et Feign pour la communication entre eux. Ce travail nous a permis de comprendre comment concevoir des systèmes distribués modernes.